AARHUS UNIVERSITY
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Term:	FALL 2024
Examination:	Declarative Programming
Date:	03 January 2025
Duration:	3 hours

- This is the exam for the lecture Declarative Programming. It consists of 18 questions. You have 3 hours to answer all of them. Strategically, you should first scan the exam paper quickly in its entirety, and identify the groups of questions that seem easiest to you. ***You should answer those first*** and then move on to the others.
- For each question it is indicated how and where the answer must be provided. There are questions that require writing a calculation or the result of a calculation, and questions that require you to write Ocaml or Prolog code, in particular. Only what is written or drawn inside the specified areas is considered as your answer.
- **If you are in doubt about what is expected, please ask!**
- Each question has a weight **[w]**, where **w** is an integer number, specifying the number of points awarded for that question. You can find the weight at the top right corner of each question. The total number of points of all questions together is 100.

Question 1 **[6]**

Recall the signature of the functions (+) and (<)

```
val (+) : int -> int -> int
val (<) : 'a -> 'a -> bool
```

What are the signatures of the functions `f`, `g` and `h`?

```
let f x y z = y
let g x y z = x + y < x
let h x y z = x < y && y < z && z < x + x
```

Answer:

`val f :` 'a -> 'b -> 'c -> 'b

`val g :` int -> int -> 'a -> bool

`val h :` int -> int -> int -> bool

Question 2 **[4]**

(a) Define a function with signature `val mean : int -> int -> int -> int` that yields the integer mean of three input values, $(a + b + c)/3$, e.g.,

```
2 = mean 1 2 3
2 = mean 2 1 4
```

Answer: `let` mean a b c = (a + b + c) / 3

(b) Define a function with signature

```
val f : (float -> float) -> float -> float
```

that takes two inputs a, b and returns a function that takes one input x and yields $(b - a) * x$, e.g.:

```
 50. =  (f 5. 10.) 10.
-50. =  (f 10. 5.) 10.
```

Answer: `let` f a b = fun x -> (b -. a) *. x

Question 3 **[6]**

Complete the following recursive function `move es` so that it implements the following behaviour:

```
move ["left"; "left"] 0 = -2
move ["right"; "left"] 10 = 10
move ["left"; "mirror"] 10 = -9
```

```
let rec move es loc =
  match es with
    [] -> loc
    | "left"::es' -> move es' (loc - 1)
    | "right"::es' -> move es' (loc + 1)
    | "mirror"::es' -> move es' (loc * -1)
```

Question 4 **[6]**

You are given the following tail recursive implementation of function `zipcode xs ys` that computes a distance for the lists `xs` and `ys`. E.g.,

```
zipcode [1] [1] = 0
zipcode [1] [2] = 1
zipcode [1; 1] [2; 3] = 3
```

```
let rec zipcode xs ys =
  match (xs, ys) with
    (x::xs', y::ys') -> (y - x) + zipcode xs' ys'
  | (_, _) -> 0
```

Complete the tail recursive function `zipcode_tr xs ys ac` below that behaves just as the function `zipcode xs ys` such that `zipcode_tr xs ys 0 = zipcode xs ys`. Hint: The argument `ac` is the accumulator.

```
let rec zipcode_tr xs ys ac =
  match (xs, ys) with
    (x::xs', y::ys') -> zipcode_tr xs' ys' (ac + y - x)
  | (_, _) -> ac
```

Question 5 **[4]**

Functions `separate`, `choose`, and `even` are defined as

```
let rec separate bs xs ys =
  match (xs, ys, bs) with
    (x::xs', _::ys', true::bs') -> x::separate bs' xs' ys'
  | (_::xs', y::ys', false::bs') -> y::separate bs' xs' ys'
  | (_, _, _) -> []

let choose f n =
  let rec aux m = if m = n then [] else f m::aux (m+1) in aux 0

let even x = x mod 2 = 0
```

(a) Using functions `separate` and `choose`, implement function `sepleft xs ys` that returns the first 10 elements of `xs`. Hint: You need lambda abstraction to implement this.
Answer:

```
let sepleft = seperate (choose (fun _ -> true) 10) xs ys
```

(b) Using functions `separate`, `choose`, and `even`, implement a function `sepeven xs ys` that returns a list `zs = [zs!0; zs!1; ...]` where `zs!i = xs!i` for even indices `i`, and `zs!i = ys!i` for uneven indices `i`, for the first 10 indices `i`. (For a list `ls` the expression `ls!k` denotes the value stored at index `k` of list `ls` starting at `0`.)
Answer:

```
let bndmapzip = seperate (choose even 10) xs ys
```

Question 6 **[7]**

(a) Unnest the following function `on`

```
let on w =
  let ith x y =
    let it u = w - u * y in
    it x in
  w + ith (1 + w) w
```

Answer:

`let it` u w y = w - u * y

`let ith` x y w = it x w y

`let on` w = w + ith (1 + w) w w

(b) Unnest the following function `part`

```
let part g x =
  let n t v =
    let parcel f = g (f v) in
    parcel (fun _ -> g t) in
  n 1 x
```

Answer:

`let parcel` f g v = g (f v)

`let n` t v g = parcel (fun _ -> g t) g v

`let part` g x = n 1 x g

Hint: Function declarations can be nested, that is, “inner” functions are declared locally to “outer” functions. Parameters to outer functions can be referred to in inner functions.
Nested functions can be unnested by declaring all “inner” functions globally before the outer function making the passed parameters explicit. The behaviours of the nested and corresponding unnested functions are indistinguishable.

Question 7 **[5]**

Given the type `prop` below

```
type prop = T | F | N of prop | O of prop * prop
```

and the recursive function `simp`

```
let rec simp p =
  match p with
    T | N F | O (T, _) -> T
  | F | N T -> F
  | N q -> simp (N (simp q))
  | O (F, r) -> simp r
  | O (q, r) -> simp (O (simp q, r))
```

We can compute

```
simp (N (O (F, N F))) = F
```

Reading the function definitions as a system of equations as we have done in the course, calculate the result of the computation by inserting the missing steps:

```
  simp (N (O (F, N F)))
= simp (N (simp (O (F, N F))))
= simp (N (simp (N F)))
= simp (N T)
=
=
= F
```

Question 8 **[5]**

(a) Given the definitions of the functions `eval` and `transform` given by

```
let eval f xs =
  match xs with
    [] -> (f 0, [])
  | x::xs' -> (f x, xs')

let rec transform f g xs =
  match eval f xs with
    (x, []) -> x
  | (x, xs') -> g x (transform f g xs')
```

implement a function `compsum` using function `transform` that adds up all elements of a list

```
0 = compsum []
0 = compsum [0]
3 = compsum [1; 2]
6 = compsum [1; 2; 3]
```

Answer:

`let compsum =` transform (fun x -> x) (+)

Note: You are not allowed to add parameters before the `=`.
Hint: You need lambda abstraction to implement this.

(b) Using `transform` implement a function `comprev xs` that returns `[0]` for the empty list and the reverse of `xs` otherwise

```
[0] = comprev []
[0] = comprev [0]
[1] = comprev [1]
[2; 1] = comprev [1; 2]
[3; 2; 1] = comprev [1; 2; 3]
```

Answer:

`let comprev =` transform (fun x -> [x]) (fun x y -> y @ x)

Note: You are not allowed to add parameters before the `=`.

Question 9 **[8]**

Given the type of formulas

```
type form = Imp of form * form | Not of form | Id of int
```

the following functions `pos` and `neg` extract positive resp. negative literals from a formula and return them in a list.

```
let rec pos fm xs =
  match fm with
    Not fm' -> neg fm' xs
  | Imp (fm', fm'') -> neg fm' (pos fm'' xs)
  | Id x -> x::xs
and neg fm xs =
  match fm with
    Not fm' -> pos fm' xs
  | Imp (fm', fm'') -> pos fm' (neg fm'' xs)
  | Id _ -> xs
```

Considering the function definitions as a system of equations, calculate:

(a) The value of `neg (Imp (Id 2, Id 1)) [1]`

`neg (Imp (Id 2, Id 1)) [1]`

= pos (Id 2) (neg Id 1 [1])

= pos (Id 2) [1]

= [2; 1]

(b) The value of `pos (Not (Imp (Id 1, Imp (Id 2, Id 3)))) []`

`pos (Not (Imp (Id 1, Imp (Id 2, Id 3)))) []`

= neg (Imp (Id 1, Imp (Id 2, Id 3))) []

= pos (Id 1) (neg Imp (Id 2, Id 3) [])

= pos (Id 1) (pos (Id 2) (neg Id 3 []))

= pos (Id 1) (pos (Id 2) [])

= pos (Id 1) [2]

= [1; 2]

Question 10 **[6]**

Consider the pattern matching algorithm match from the course

$$
\begin{array}{lll}
\mathrm{match}\ S\ u\ =\ u & & \text{if } S = \emptyset \\
\qquad \mathrm{match}\ (S \setminus (x,t))\ u & & \text{if } (x,t) \in S \cap u \\
\qquad \mathrm{match}\ (S \setminus (x,t))\ (u \cup \{(x,t)\}) & & \text{if } (x,t) \in S \wedge (x,s) \notin u \text{ for all } s \\
\qquad \mathrm{match}\ (S \cup \{(s_1,s'_1),\ldots,(s_k,s'_k)\}\ u & & \text{if } (c(s_1,...,s_k), c'(s'_1,...,s'_k)) \in S \wedge c = c' \\
\qquad \mathbf{fail} & & \text{otherwise}
\end{array}
$$

Let v and h denote variables and X, Y, P, T, U and Z constants. As usual, we leave away empty argument lists, e.g., instead of $Z()$ we write Z.

Calculate:

(a) The value of $\mathrm{match}\ \{(X(Y(v,P(h),U),T(v)),\ X(Y(Z,P(P(Z)),U),T(Z))\}\ \emptyset$

$\mathrm{match}\ \{(X(Y(v,P(h),U),T(v)),\ X(Y(Z,P(P(Z)),U),T(Z))\}\ \emptyset$

$=$ match {(Y(v, P(h), U), Y(Z, P(P(Z)), U), (T(v), T(Z))} Ø

$=$ match {(v, Z), (P(h), P(P(Z))), (U, U), (T(v), T(Z)} Ø

$=$ match {(P(h), P(P(Z))), (U, U), (T(v), T(Z))} {(v, Z)}

$=$ match {(h, P(Z)), (U, U), (T(v), T(Z))} {(v, Z)}

$=$ match {(U, U), (T(v), T(Z))} {(v, Z), (h, P(Z))}

$=$ match {(T(v), T(Z))} {(v, Z), (h, P(Z))}

$=$ match {(v, Z)} {(v, Z), (h, P(Z))}

$=$ match Ø {(v, Z), (h, P(Z))}

$=$ {(v, Z), (h, P(Z))}

(b) The value of $\mathrm{match}\ \{(X(Y(U,v)),X(Y(P,Z)))\}\ \emptyset$

$\mathrm{match}\ \{(X(Y(U,v)),X(Y(P,Z)))\}\ \emptyset$

$=$ match {(Y(U, v), Y(P, Z))} Ø

$=$ match {(U, P), (v, Z)} Ø

$=$ fail

Question 11 **[5]**

Using the truth table method, determine whether the formula

$$(P \Rightarrow Q) \wedge \neg P$$

is satisfiable.

Fill in the truth table:

P	Q	$(P \Rightarrow Q) \wedge \neg P$
T	T	F
T	F	F
F	F	T
F	T	T

Tick the correct answer:

☒ The formula is satisfiable.

☐ The formula is unsatisfiable.

Question 12 **[4]**

Determine whether the formula

$$\begin{array}{ll} & Q \vee R \\ \wedge & \neg Q \vee \neg R \\ \wedge & P \vee \neg Q \vee \neg R \\ \wedge & \neg R \end{array}$$

is satisfiable applying the DPLL algorithm, using the rules

(UR) the 1-literal rule (aka unit clause rule),
(PR) the affirmative-negative rule (aka pure literal rule),
(SR) the splitting rule.

Fill into the boxes the formulas resulting from the rule applications indicated between the boxes.

1:

$$\begin{array}{ll} & Q \vee R \\ \wedge & \neg Q \vee \neg R \\ \wedge & P \vee \neg Q \vee \neg R \\ \wedge & \neg R \end{array}$$

Applying rule **(PR)** for P to the formula in box 1 yields:

2:

Applying rule **(UR)** to the formula in box 2 yields:

3: Q

Applying rule **(UR)** to the formula in box 3 yields:

4: T

Tick the correct answer:
☒ The formula is satisfiable.
☐ The formula is unsatisfiable.

Question 13 **[6]**

The following table specifies two lambda terms in the first row. Complete the empty cells in the remaining rows of the table according to the lambda terms for each column.
Remark: In your answers you may write "L" to represent "λ" if you prefer.

Lambda term	$\lambda a \ \cdot (\lambda b \cdot f\ x\ a)$	$\lambda x\ y \cdot (y\ (\lambda g \cdot x\ x))$
Name the **type** of lambda term	abstraction	abstraction
Rewrite the term in its fully curried form		
Specify the **free** variables	f, x	none
Specify the **bound** variables	a, b	x, y, g

Question 14 **[6]**

A turnstile gate controls access to a museum in Aarhus. When the turnstile gate is locked, people cannot go through (i.e. pushing on the gate barrier has no effect). When a coin is deposited, the turnstile gate unlocks and allows one person to pass through (i.e. they push the gate, the gate rotates once allowing one person to enter, and then the gate locks again). This is modelled as a finite state machine in the following Prolog program, where $transition(A, S1, S2)$ specifies a state transition when the action A occurs in state $S1$ transitioning the system into state $S2$:

```
state(locked).
state(unlocked).

transition(coin, unlocked, unlocked).
transition(coin, locked,   unlocked).

transition(push, locked,   locked).
transition(push, unlocked, locked).

start(locked).
```

(a) For each of the following Prolog queries applied to the above program, write down the first query result when executed in Prolog. For example:

```
?- state(S).
```

First query result when executed in Prolog:

```
S = locked.
```

(i)
```
?- state(S2), transition(A, unlocked, S2).
```

First query result when executed in Prolog:

S2 = locked, A = push.

(ii)
```
?- start(S1), transition(push, S1, S2),
   transition(A2, S2, unlocked).
```

First query result when executed in Prolog:

S1 = locked, S2 = locked, A2 = coin.

(b) Given a list of actions, we want to determine the sequence of states that the finite state machine transitions through. For example, starting from "locked", given the action sequence:
`[coin, push, push]`
. . . the state machine transitions through the states:
`[locked, unlocked, locked, locked]`

Hint: Recall that, in Prolog, list terms have the form *[Head | Tail]* and *[]* for the empty list, the relation `A = B` is true when term A unifies with term B, and the relation `A \= B` is true when two terms A, B can not unify.

(i) Complete the following two Prolog rules to define the predicate `seq(Actions, S, States)`, such that `Actions` is a list of actions, `S` is the current state, and `States` is the sequence of states transitioned through after state `S`. For example, the query:
`?- seq([coin, push, push], locked, States).`
. . . should give the result:

`States = [locked, unlocked, locked, locked]`

```
seq([], S1, L) :-  L = [S1].

seq([A|ARest], S1, [S1|SRest]) :-
    Transition(A, S1, S2), seq(ARest, S2, SRest).
```

(ii) Use the `seq` predicate in a query to find the sequence of actions (if any) that result in the given sequence of states defined by the state machine, starting from the state defined by the `start/1` fact:

```
?- States = [locked, locked, unlocked],
 start(S), seq(Actions, S, States).
```

(iii) If a finite state machine is *deterministic* then every state has exactly one transition for each possible input. Write a query to determine whether the turnstile state machine defined in the Prolog program is deterministic.

```
?-  transition(A, S, S1), transition(A, S, S2), S1 \ = S2.
```

Question 15 **[4]**

(a) Write down the first result of each of the following Prolog queries that use built-in list predicates.

```
?- member(S, [1,3,5]), not(S < 4).
```

First query result when executed in Prolog:

S = 5.

```
?- append(_, [X], [a,b,c,d]).
```

First query result when executed in Prolog:

X = d.

(b) Write down the result of executing the following unification queries in Prolog.

```
?- file_type(fxt, [2,7,1], A0) =
   file_type(Typ, [F|L], [Typ|[F]]).
```

First query result when executed in Prolog:

Typ = fxt, F = 2, L = [7, 1], A0 = [fxt, 2].

```
?- [ [mars, venus], [saturn, jupiter] ] = [H|T].
```

First query result when executed in Prolog:

H = [mars, venus], T = [[saturn, jupiter]].

Question 16 **[8]**

(a) Two sorted lists of integers L_x, L_y can be *merged* into one sorted list containing all elements of both L_x and L_y. For example, the sorted lists $[1, 3, 5]$ and $[2, 4, 6]$ can be merged into the sorted list $[1, 2, 3, 4, 5, 6]$. The rules for merging two sorted lists are as follows:

- if L_x is an empty list, then L_y is the merged sorted list
- if L_y is an empty list, then L_x is the merged sorted list

Otherwise, neither L_x nor L_y are empty. Let x be the head of list L_x and xs be the tail of L_x, and similarly let y be the head of list L_y and ys be the tail of L_y, then:

- If $x \leq y$ then the merged sorted list has head x and tail m, where m is the merged sorted list of xs with L_y
- If $y < x$ then the merged sorted list has head y and tail m, where m is the merged sorted list of L_x and ys

Use this definition to implement the predicate $merge/3$ such that $merge(LX, LY, M)$ is True when M is a sorted list containing all elements of sorted lists LX and LY. Write recursive Prolog rules to define the predicate $merge/3$.

Hint: You will need to implement Prolog facts or rules for the base cases, and Prolog rules for the recursive cases.

Answer:

(b) The *merge* predicate is used to sort a list of numbers L in the $merge_sort$ algorithm. For example, the list of numbers $[3, 5, 1, 4, 2, 6]$ is sorted in the list $[1, 2, 3, 4, 5, 6]$ by following the rules below:

- If L is empty, or L has exactly one element, then the sorted list is L.
- Otherwise, split L into two sublists A, B (i.e. L is the concatenation of A and B), where A and B have the same length (or A has one element less than B if the length of L is odd). Recursively apply $merge_sort$ to A to get A_S, and recursively apply $merge_sort$ to B to get B_S, and then *merge* A_S and B_S. The result is a sorted list containing all elements of L.

Use this definition to implement the predicate $merge_sort/2$ such that $merge_sort(L, S)$ is True when S is a sorted list containing all integers in the list L. Write recursive Prolog rules to define the predicate $merge_sort/2$. You can use the predicate $half_as_long(A, L)$ defined as follows:

```
half_as_long(A, L) :-
  length(L, LenL),
  LenA is floor(LenL / 2),
  length(A, LenA).
```

Hint: You will need to implement Prolog facts or rules for the base cases, and a Prolog rule for the recursive case. You should use built-in predicates such as $append$, and use the $merge/3$ predicate you defined above.

Answer:

Question 17 **[4]**

In the lectures we discussed the unification procedure, consisting of six rewriting "rules" that step-by-step transform a set of unification expressions until arriving at a "solved substitution" or a failure.

The six rules are: *swap*, *eliminate*, *decompose*, *conflict*, *occurs*, *delete*.

Apply the same procedure to unify the expressions in (a) and (b) below. Apply exactly one "rule" at each step, and specify the rule used by writing the rule name to the right side of the new unification expression set (as we did in lectures).
Remark: In your answers you can write "$=$" to represent "$\approx$" if your prefer.

(a) Unify the term `a(A,b(b(a),c(B)))` with the term `a(c(b),b(C,c(a(b))))`.

Answer:

$\{a(A, b(b(a), c(B))) \approx a(c(b), b(C, c(a(b))))\}$

$\rightarrow$ _____

(b) Unify the term `cal(date(day(K),W,year(Z)), time(hour(17),min(X)))` with the term `cal(date(day(X),month(jan),year(2067)), time(Y,min(3)))`.

Answer:

$\{cal(date(day(K), W, year(Z)), time(hour(17), min(X))) \approx$

$cal(date(day(X), month(jan), year(2067)), time(Y, min(3)))\}$

$\rightarrow$ _____

Question 18 **[6]**

We are creating a simple task scheduling application. A task has a start time S, and an end time E. We define three binary relations between tasks:

- a task can take place *before* another task
- a task can take place *during* another task
- a task can *overlap* another task

Consider the following Prolog program that uses the CLP(FD) library for arithmetic operators:

```
tasks([]).
tasks([task(S,E)|R]) :-
  S in 1..100, E in 1..100,
  S #< E,
  tasks(R).

relation(before, task(_,E1), task(S2,_)) :-
  E1 #< S2.

relation(during, task(S1,E1), task(S2,E2)) :-
  S2 #< S1, E1 #< E2.

relation(overlaps, T1, T2) :-
  tasks([J]),
  relation(during,J, T1),
  relation(during,J, T2).
```

(a) For each of the following Prolog queries applied to the above program, write down the first query result when executed in Prolog, or write down the exception that is thrown.

(i) `?- A = task(5,15), B = task(10, 20), relation(R,A,B).`

First query result or exception when executed in Prolog:

R = overlaps.

(ii)
```
?- tasks([A,B,C]), relation(during,A,B),
   relation(before,B,C), relation(overlaps,A,C).
```

First query result or exception when executed in Prolog:

false.

(iii)
```
?- tasks([A,B]),
   relation(R1,A,B), relation(R2,A,B), R1 \= R2.
```

First query result or exception when executed in Prolog:

R1 = during, R2 = overlaps.